AUTOMATED WITHDRAWAL INTEGRATION

Mobile App, Provider Web and Admin Frontend API Handoff

Prepared for	Prepared by
HustleApp Mobile App Team **Provider Web Frontend Team** **Admin Frontend Team**	Stitchitin Solution World Limited Innovate • Build • Deliver

Release: Paystack Payouts 2026-07-17 | API Version: 1.1.0 | Handoff Version: 1.0 | 23 July 2026

Purpose of this handoff

This document gives the mobile and frontend teams the exact screens, API calls, status rules, admin controls, retry behaviour, security boundaries and acceptance tests required to support Paystack-powered provider withdrawals without exposing Paystack credentials to any client application.

CONFIDENTIAL - TECHNICAL IMPLEMENTATION DOCUMENT

1. Current Readiness and Implementation Position

The backend release for automated Paystack withdrawals has been installed and verified. The mobile and frontend teams can now implement the new withdrawal journey against the live HustleApp API while the backend remains protected by disabled payout feature flags until the controlled launch test.

Area	Current position	Frontend implication
Backend and database	Payout code installed; database migration verified; PHP and Paystack preflight passed.	Use the endpoints and states in this document. Do not reproduce payout logic in the client.
Paystack dashboard	Live webhook and live Transfer Approval URLs configured. Transfer confirmation OTP remains ON for the first controlled test.	Admin frontend must support the merchant OTP stage when a withdrawal reaches status otp.
Queue worker	Manual execution succeeded with processed=0.	Before go-live, backend team must confirm the one-minute production schedule is active.
Payout activation	PAYSTACK_PAYOUTS_ENABLED=false and PAYSTACK_PAYOUTS_AUTO_PROCESS=false.	UI development and API testing can proceed, but no team should describe a withdrawal as automatically paid until backend activation.
Security	Live secret key remains server-side. IP whitelist and 2FA are client technical controls.	Never request, store, log or display Paystack secret keys in mobile, React or admin JavaScript.

> **Launch position**
>
> The teams are integration-ready, not yet fully payout-live. Production automation is activated only after the backend owner confirms the queue schedule, IP whitelist, controlled GHS test and final feature-flag change.

Responsibility split

Team	Required responsibility
Mobile app team	Implement provider payout-bank setup, account resolution, withdrawal request, provider OTP confirmation, wallet refresh, status display and user-safe errors.
Provider web frontend team	Implement the same provider flow on web using the same API contract and status copy.
Admin frontend team	Implement withdrawal list/detail, status-aware action buttons, Paystack merchant OTP modal, verification, retry, decline and manual-review actions.

Team	Required responsibility
Backend team	Own all Paystack calls, recipient creation, transfer references, wallet reservations/refunds, webhooks, queue processing, reconciliation and feature flags.
Client technical team	Own Paystack dashboard configuration, IP whitelist, 2FA, merchant OTP contact and live configuration sign-off.

2. Shared API Contract and Non-Negotiable Rules

Item	Value
Base URL	https://api-v2.hustleapp.info/api/v1
Authentication	Authorization: Bearer <authenticated-user-token>
Content type	application/json
Provider role	Authenticated artisan/provider only for /wallet routes
Admin role	Authenticated authorised admin only for /admin/withdrawals routes
Currency at launch	GHS
Payout rail	Ghana bank account through Paystack GIP/ghipss recipient configuration

Client-generated idempotency key

For bank-account creation and withdrawal creation, send X-Idempotency-Key. Generate one UUID for the user action and reuse the same value when retrying the same request after a timeout. Generate a new value only when the user starts a genuinely new action.

```
X-Idempotency-Key: 8f35b7ab-8b16-4dc0-9ba3-b85bd88b8ef4
```

> **Do not double-submit**
>
> Disable the submit button after the first tap/click. Never create a fresh idempotency key for an automatic retry of the same withdrawal because that may represent a new request.

Two different OTPs must not be confused

OTP	Who receives it	Where it is entered	Purpose
HustleApp withdrawal OTP	The provider, through the provider account email	Mobile app or provider web	Confirms that the provider requested the withdrawal.

OTP	Who receives it	Where it is entered	Purpose
Paystack merchant transfer OTP	The client's authorised transfer contact	Admin frontend only, when status is otp	Confirms the outgoing Paystack transfer during the controlled rollout stage.

- The provider must never see or enter the Paystack merchant OTP.
- The mobile app must never call Paystack directly.
- The admin frontend must never create a transfer reference or manually mark a withdrawal paid.

3. Provider Journey - Bank Account Setup

Step 1 - Load supported payout banks

```
GET /wallet/payout-banks?country=ghana&currency=GHS&type=ghipss
Authorization: Bearer <provider-token>
```

Populate a searchable bank picker using data.items. Use the returned code as bank_code. Do not hard-code the bank list and do not allow a free-text bank name to become the trusted value.

```
{
  "success": true,
  "message": "Payout banks loaded.",
  "data": {
    "items": [
      {
        "id": 123,
        "name": "Example Bank Ghana",
        "code": "030100",
        "slug": "example-bank-ghana",
        "country": "Ghana",
        "currency": "GHS",
        "type": "ghipss"
      }
    ]
  }
}
```

Step 2 - Resolve the account before saving

```
POST /wallet/bank-accounts/resolve
Authorization: Bearer <provider-token>
Content-Type: application/json

{
  "bank_code": "030100",
  "account_number": "0123456789"
}
```

Show the returned account_name, bank_name and masked_account_number on a confirmation screen. The account holder name is Paystack-resolved and must be read-only.

```
{
  "success": true,
  "message": "Bank account resolved.",
  "data": {
    "account_number": "0123456789",
    "masked_account_number": "******6789",
    "account_name": "RESOLVED ACCOUNT NAME",
    "bank_code": "030100",
    "bank_name": "Example Bank Ghana",
    "currency_code": "GHS",
    "recipient_type": "ghipss"
  }
}
```

Step 3 - Save the verified payout account

```
POST /wallet/bank-accounts
Authorization: Bearer <provider-token>
X-Idempotency-Key: <stable-uuid-for-this-save>
Content-Type: application/json

{
  "bank_code": "030100",
  "account_number": "0123456789",
  "is_default": true
}
```

The backend resolves the account again, creates or reuses the Paystack recipient and returns data.item. Save item.id as payout_bank_account_id. A payout-ready account must have payout_ready=true, verification_status=verified and is_active=1.

4. Provider Journey - Withdrawal Request and Confirmation

Step 4 - Load saved payout accounts

```
GET /wallet/bank-accounts?page=1&per_page=20
Authorization: Bearer <provider-token>
```

- Show only masked_account_number or account_number_last4; never display the raw stored account number after save.
- Allow selection only when payout_ready=true and is_active=1.
- Show verified account name from resolved_account_name or beneficiary_name returned by the API.

Step 5 - Load wallet balance

```
GET /wallet
Authorization: Bearer <provider-token>
```

Use available_balance as the maximum amount the user may request. Do not calculate the final allowed amount only on the client; the backend remains authoritative for minimum, maximum, daily limit, KYC, currency and available balance.

Step 6 - Request a withdrawal

```
POST /wallet/withdrawals
Authorization: Bearer <provider-token>
X-Idempotency-Key: <stable-uuid-for-this-withdrawal>
Content-Type: application/json

{
  "amount": "10.00",
  "payout_bank_account_id": 123
}
```

```
{
  "success": true,
  "message": "Withdrawal requested. Verify with the OTP sent to your email to continue.",
  "data": {
    "withdrawal_id": 456,
    "status": "pending",
    "amount": "10.00",
    "currency_code": "GHS",
    "withdrawal_otp_email_sent": true
  }
}
```

Navigate to the provider OTP screen only after HTTP 201. Keep withdrawal_id in the screen state. In production, otp_code is null and must never be expected from the response.

Step 7 - Verify the provider's HustleApp OTP

```
POST /wallet/withdrawals/456/verify-otp
Authorization: Bearer <provider-token>
Content-Type: application/json

{
  "otp_code": "123456"
}
```

```
{
  "success": true,
  "message": "Withdrawal confirmed and awaiting administrator approval.",
  "data": {
    "withdrawal_id": 456,
    "status": "approved"
  }
}
```

> **Expected launch response**
>
> The recommended launch mode requires admin approval, so successful provider OTP verification returns approved. If full automation is enabled later, the same endpoint may return

queued. The app must support both.

5. Status Lifecycle and Required Display Copy

The API status is the source of truth. Never translate an intermediate status into “Paid”. A withdrawal is completed only when the API reports paid after Paystack’s final transfer event or a verified reconciliation.

API status	Provider display	Admin display	Final?	Recommended UI
pending	Awaiting OTP confirmation	Provider OTP pending	No	Show OTP entry/resume action.
approved	Awaiting payout approval	Ready for approval	No	Provider sees processing timeline; admin sees Approve and Decline.
queued	Payout queued	Queued for Paystack	No	Disable duplicate actions; allow admin Verify/Manual Review.
processing	Payout is being processed	Paystack processing	No	Show neutral progress, not success.
otp	Payout confirmation in progress	Merchant transfer OTP required	No	Provider waits; admin opens merchant OTP modal.
manual_review	Payout under review	Manual reconciliation required	No	Show support/review message; admin Verify first.
paid	Payout completed	Paid	Yes	Show success receipt/timestamp where supplied.
failed	Payout failed; funds returned	Failed and refunded	Yes	Show wallet-refund message and refresh wallet.
reversed	Payout reversed; funds returned	Reversed and refunded	Yes	Show wallet-refund message and refresh wallet.
declined	Withdrawal declined; funds returned	Declined and refunded	Yes	Show reason where supplied and refresh wallet.

Lifecycle

Provider action	Backend/admin stage	Paystack stage	Final outcome
Request → provider OTP	pending → approved → queued	processing → otp (during controlled rollout) or processing	paid OR failed/reversed/declined with refund

> **Current Paystack configuration**
>
> Because “Confirm transfers before sending” is currently enabled in the client’s Paystack dashboard, the first live transfer is expected to enter otp. This is normal. The admin frontend must provide the Paystack merchant OTP action.

Provider-facing status refresh

Provider clients must not call admin routes. For this release, use HustleApp notifications plus GET /wallet and GET /wallet/entries?entry_type=withdrawal after a withdrawal update. A separate provider withdrawal-detail route is not part of this release; do not attempt to access /admin/withdrawals with a provider token.

6. Mobile App Implementation Requirements

Screen/component	Required behaviour
Payout account list	GET saved accounts; show bank name, verified account name, masked number, default badge and payout-ready state.
Add payout account	Load bank picker from API, accept account number, resolve, show confirmation, then save with idempotency key.
Withdrawal form	Show available balance, GHS currency, amount field, selected verified account and backend validation messages.
Provider OTP screen	Six-digit input, expiry guidance, submit once, prevent automatic duplicate submission and retain withdrawal_id.
Withdrawal result	Display returned approved or queued state using the status copy in Section 5. Do not show a payment-success screen yet.
Wallet activity	Refresh wallet and withdrawal entries after notifications, app resume and final-state messages.
Notifications	Open the appropriate wallet/withdrawal context using withdrawal_id and status from the notification payload.
Offline/network handling	Keep the same idempotency key when retrying the same create-account or withdrawal request after a timeout.

Mobile validation guidance

- Trim spaces from the account number before sending, but preserve it as a string.
- Send withdrawal amount as a decimal string, for example "10.00", not a floating-point calculation result.
- Do not allow manual entry of beneficiary/account name.
- Do not hard-code the GHS 10 minimum as the only validation source; show backend messages because limits can change.
- Do not store Paystack recipient codes in a place visible to the user; use payout_bank_account_id for client selection.

Shared TypeScript status map (React Native/Expo and React)

```
export const withdrawalStatusCopy: Record<string, string> = {
  pending: 'Awaiting OTP confirmation',
  approved: 'Awaiting payout approval',
  queued: 'Payout queued',
  processing: 'Payout is being processed',
  otp: 'Payout confirmation in progress',
  manual_review: 'Payout under review',
  paid: 'Payout completed',
  failed: 'Payout failed; funds returned',
  reversed: 'Payout reversed; funds returned',
  declined: 'Withdrawal declined; funds returned',
};
```

7. Provider Web Frontend Requirements

The provider web frontend must mirror the mobile flow and use the same API contract. Do not maintain a separate bank list, beneficiary-name workflow or payout status interpretation.

Area	Implementation requirement
Responsive forms	Bank picker, account resolve confirmation, withdrawal form and OTP modal must work on mobile-width web views.
Submit protection	Disable buttons while requests are in flight; preserve the idempotency key across a timeout retry.
Account confirmation	Require the user to confirm the Paystack-resolved account name before the save request.
Sensitive display	Mask account numbers everywhere after creation. Do not render recipient codes, provider payloads or raw Paystack fields.
State persistence	Keep withdrawal_id through route changes until provider OTP completes; clear it after a successful confirmation or cancellation.

Area	Implementation requirement
Status components	Use one shared status-badge component based on Section 5. Never treat HTTP 200 from an admin action as proof of bank payment.
Accessibility	Label amount, bank, account number and OTP fields; provide text status in addition to colour.

Recommended request wrapper

```
async function createWithdrawal(input, idempotencyKey, token) {
  const response = await fetch(`${API_BASE}/wallet/withdrawals`, {
    method: 'POST',
    headers: {
      Authorization: `Bearer ${token}`,
      'Content-Type': 'application/json',
      'X-Idempotency-Key': idempotencyKey,
    },
    body: JSON.stringify({
      amount: input.amount,
      payout_bank_account_id: input.payoutBankAccountId,
    }),
  });

  const payload = await response.json();
  if (!response.ok) throw new Error(payload.message || 'Withdrawal request failed.');
  return payload.data;
}
```

> **Client error rule**
>
> Show the API message to the user when it is safe and actionable. Log request_id or API metadata for support, but never log the bearer token, account number, OTP or secret key.

8. Admin Frontend - Withdrawal Operations

List and detail routes

```
GET /admin/withdrawals?status=approved&page=1&per_page=20
GET /admin/withdrawals/{withdrawal_id}
Authorization: Bearer <admin-token>
```

The withdrawal detail should show provider, amount, currency, status, bank name, resolved account name, masked account number, transfer reference, Paystack transfer code, provider status/message, retry count and relevant timestamps. Do not display the raw account number or secret key.

Status-aware action matrix

Current status	Primary actions	Actions that must not appear
pending	View only; provider has not completed OTP.	Approve, mark paid, retry.
approved	Approve and queue; Decline with reason.	Manual mark paid.
queued / processing	Verify; Manual review.	Approve again; Decline after transfer exists.
otp	Finalize merchant OTP; Verify; Manual review.	Provider OTP input; manual mark paid.
failed	Verify first; Retry only after backend confirms safe state.	Blind retry; manual mark paid.
manual_review	Verify; Retry only after reconciliation; Decline only when no transfer exists.	Automatic assumption of failure or success.
paid / reversed / declined	Read-only.	All mutation actions.

Dedicated admin actions

Action	Endpoint	Request body
Approve and queue	POST /admin/withdrawals/{id}/approve	None
Verify with Paystack	POST /admin/withdrawals/{id}/verify	None
Finalize Paystack OTP	POST /admin/withdrawals/{id}/finalize-otp	{ "otp": "123456" }
Retry safe payout	POST /admin/withdrawals/{id}/retry	None
Decline and refund	POST /admin/withdrawals/{id}/decline	{ "failure_reason": "Reason" }
Manual review	POST /admin/withdrawals/{id}/manual-review	Optional reason through compatible action route if supported

9. Admin Merchant OTP Flow for the Current Launch Stage

The client currently has Paystack transfer confirmation enabled. Therefore, the admin frontend must support a second OTP that is sent by Paystack to the client's transfer contact after the queue worker initiates the transfer.

1. Admin approves an approved withdrawal. POST /admin/withdrawals/{id}/approve returns a queued result.
2. The queue worker initiates the Paystack transfer. The withdrawal may become otp.
3. Admin detail/list refresh detects status=otp and displays "Enter Paystack transfer OTP".
4. An authorised client representative supplies the Paystack OTP received by email.
5. Admin submits the OTP through the finalize endpoint.
6. The UI remains processing until Paystack sends transfer.success or the admin verifies the transfer.

```
POST /admin/withdrawals/456/finalize-otp
Authorization: Bearer <admin-token>
Content-Type: application/json

{
  "otp": "PAYSTACK_OTP"
}
```

> **Do not mark paid after OTP submission**
>
> A successful finalize-otp response only confirms that the OTP was submitted. The transfer remains non-final until the webhook or Paystack verification changes the withdrawal to paid.

Later automation stage

After the controlled live test and written client sign-off, Paystack transfer OTP may be switched OFF while Transfer Approval remains ON. The admin frontend should keep the otp UI available for rollback or configuration changes, but it will normally not appear.

10. Error Handling and Safe Retry Behaviour

HTTP / condition	Meaning	Client behaviour
401	Missing/expired authentication or invalid webhook signature.	User routes: refresh login/session. Never retry with the same expired token.
403	Provider KYC is not approved or role is not permitted.	Show KYC requirement or access message; do not bypass.
404	Payout account or withdrawal was not found for that user.	Return to list, refresh data and do not expose IDs from another account.
422	Validation failed: account resolution, amount, balance, OTP, currency, limits or unsafe admin transition.	Show API message; correct input or refresh state. Do not force transition.
502	Paystack could not be reached for bank/account operation.	Show temporary service message and allow a deliberate retry.
Network timeout on idempotent POST	Outcome may be unknown.	Retry with the same X-Idempotency-Key, not a new key.
Duplicate tap/click	Same action could be submitted more than once.	Disable controls and keep one in-flight request.
Webhook delay	Transfer initiation succeeded but final event has not arrived.	Keep processing state and allow admin Verify; never infer paid from elapsed time.

Known authoritative messages

- KYC approval is required before requesting withdrawals.
- Minimum withdrawal is GHS 10.00. (Current backend configuration; the API remains authoritative.)
- Maximum single withdrawal is GHS 5000.00. (Current backend configuration.)
- Daily withdrawal limit is GHS 10000.00. (Current backend configuration.)
- Insufficient wallet balance.
- Select a bank account that has been verified for Paystack payouts.
- Invalid or expired OTP.
- Manual paid status is disabled. Use Paystack verification instead.

11. Security and Data-Handling Rules

Rule	Required implementation
No direct Paystack access	Mobile, provider web and admin JavaScript call only HustleApp API routes.
Secrets stay server-side	Never request, bundle, log, display or store sk_live, webhook signature logic or Paystack dashboard credentials.
Mask bank details	After save, use masked_account_number or account_number_last4. Never render full account number in lists, logs or analytics.
Protect OTPs	Do not log provider OTP or Paystack merchant OTP. Clear OTP values after submit or navigation.
Bearer-token hygiene	Use secure client storage appropriate to the app; never print tokens in production logs or screenshots.
Role separation	Provider tokens must not call admin endpoints. Admin OTP operation must not exist in provider UI.
Status integrity	Only paid is success. Failed, reversed and declined must visibly state that funds were returned.
Audit-friendly UI	For admin actions, show confirmation prompts and preserve request/reference identifiers returned by the backend for support.

> **Prohibited legacy behaviour**
>
> Remove any admin field that accepts an arbitrary transfer reference and any button that directly sets status to paid. The backend now owns transfer references and final status.

12. Team Implementation Sequence

Sequence	Mobile / provider web	Admin frontend	Backend dependency
1	Build bank picker and account resolver.	Build withdrawal list/detail shell.	Payout bank and account endpoints available.
2	Save verified account and list masked accounts.	Add status badges and safe action matrix.	Recipient creation remains server-side.
3	Build withdrawal request and provider OTP screens.	Add Approve, Decline, Verify and Manual Review.	Feature flags may remain disabled during UI work.
4	Implement status copy, notifications and wallet refresh.	Add merchant Paystack OTP modal for status otp.	Queue worker schedule must be confirmed.
5	Run test/staging validation and duplicate-submit tests.	Run admin safe-transition and retry tests.	Backend reconciles final events and refunds.
6	Participate in controlled GHS live test.	Complete merchant OTP and observe final paid event.	Backend temporarily enables controlled mode.

Do not wait for full automation to build the UI

The recommended production launch is admin-approved automation. The app should be ready for approved, queued, processing, otp and final states before the backend feature flag is enabled. Full auto-processing is a later configuration change, not a separate client rewrite.

13. Acceptance Test Checklist

Provider app and provider web

- ☐ Bank list loads from GET /wallet/payout-banks and is searchable.
- ☐ Invalid account number produces a controlled 422 message.
- ☐ Resolved account name is displayed read-only before save.
- ☐ Verified account is returned with payout_ready=true and a masked account number.
- ☐ Duplicate save or withdrawal retry uses the same idempotency key.
- ☐ KYC, minimum, maximum, daily limit and insufficient-balance messages render correctly.
- ☐ Withdrawal creation returns pending and opens the provider OTP screen.
- ☐ Invalid/expired OTP does not change the withdrawal state.
- ☐ Successful OTP shows approved or queued, never paid.
- ☐ Final failed/reversed/declined notification refreshes wallet and shows funds returned.

Admin frontend

- ☐ Withdrawal list filters and detail view load from the new admin endpoints.
- ☐ Full bank account number and Paystack secret data are not displayed.
- ☐ Approved status shows Approve and Decline only.
- ☐ Approve action returns queued and cannot be double-clicked.
- ☐ Status otp opens the merchant OTP control, not the provider OTP control.
- ☐ Finalize OTP does not immediately display paid.

- ☐ Verify action refreshes the authoritative Paystack result.
- ☐ Failed/manual-review retry is blocked when the backend reports unsafe state.
- ☐ Paid, reversed and declined rows are read-only.
- ☐ Manual “mark paid” and arbitrary transfer-reference fields are removed.

Controlled live test

- ☐ Company-controlled Ghana bank account is used.
- ☐ Small approved GHS amount is used and Paystack balance covers amount plus fee.
- ☐ Provider OTP, admin approval, queue processing and Paystack merchant OTP are observed.
- ☐ Paystack transfer reference matches the HustleApp record.
- ☐ Final transfer.success changes the withdrawal to paid.
- ☐ Destination account receipt, Paystack dashboard, HustleApp status and wallet ledger reconcile.

14. Quick Endpoint Reference

Audience	Method and route	Purpose
Provider	GET /wallet	Wallet summary.
Provider	GET /wallet/entries?entry_type=withdrawal	Wallet withdrawal activity.
Provider	GET /wallet/payout-banks	Load supported Ghana payout banks.
Provider	POST /wallet/bank-accounts/resolve	Resolve account name through Paystack.
Provider	GET /wallet/bank-accounts	List saved payout accounts.
Provider	POST /wallet/bank-accounts	Create/reuse Paystack recipient and save verified account.
Provider	POST /wallet/withdrawals	Create withdrawal and send provider OTP.
Provider	POST /wallet/withdrawals/{id}/verify-otp	Confirm provider OTP and reserve funds.
Admin	GET /admin/withdrawals	List withdrawal and payout states.
Admin	GET /admin/withdrawals/{id}	Read withdrawal, recipient, transfer and reconciliation detail.
Admin	POST /admin/withdrawals/{id}/approve	Approve and queue.
Admin	POST /admin/withdrawals/{id}/verify	Verify/reconcile directly with Paystack.
Admin	POST /admin/withdrawals/{id}/finalize-otp	Submit Paystack merchant transfer OTP.
Admin	POST /admin/withdrawals/{id}/retry	Retry only after safe reconciliation.
Admin	POST /admin/withdrawals/{id}/decline	Decline before a transfer exists and refund.
Admin	POST /admin/withdrawals/{id}/manual-review	Place uncertain transfer under manual review.

Supporting machine-readable files

The delivery bundle also includes the OpenAPI 3.0 specification and Postman collection from the installed backend release. Import the Postman collection and set token, adminToken, bankCode, accountNumber, payoutBankAccountId, withdrawalId and withdrawalOtp.

Implementation sign-off

Role	Name	Signature	Date
Mobile App Representative			
Provider Web Frontend Representative			
Admin Frontend Representative			
HustleApp Backend Representative			

End of document